OOP, Classes and Objects

A Concept Introduction

What You'll Learn

#	Topic	The question it answers
1	What is OOP?	Why bundle data and behaviour together?
2	Benefits: Reusability, Security, Extensibility	Why do companies insist on it?
3	Class & Object	What is the actual difference?
4	Class Definition and Syntax	How do I write one properly?
5	Creating and Using Objects	What really happens when I type `new`?
6	Instance vs Class (static) Members	Who owns what?

1. What is OOP?

Start With the Mess

Let's build the attendance tracker the old way, for just two students:

```
String name1 = "Ravi";
String regNo1 = "RA2311003010123";
int attendance1 = 82;

String name2 = "Anitha";
String regNo2 = "RA2311003010124";
int attendance2 = 68;
```

Now the advisor says: *"Also check who is detained."* So you write the check. Twice. Then a third student joins. Then you need CGPA too.

Notice what is wrong here. **The computer has no idea that `name1`, `regNo1`, and `attendance1` have anything to do with each other.** You know they describe Ravi. Java sees three unrelated variables.

The Shift

Object-Oriented Programming says: *stop describing steps, start describing things.*

A student is one thing. That thing **has** a name, a registration number, and an attendance percentage. That thing **can** tell you whether it is exam-eligible. Bundle all of that into a single unit, and now the computer knows they belong together too.

```
Procedural thinking : What steps should my program perform?
OOP thinking        : What things exist, and what can each of them do?
```

You have already been doing this without noticing. Last week you wrote `city.length()`. You did not pass `city` into a `length()` function — you *asked the String itself*. That is object thinking. Java has been nudging you toward it all along.

> **WHY THIS MATTERS**
>
> - **Nothing drifts out of sync.** Ravi's attendance physically lives inside Ravi's object. There is no second array to forget about.
> - **Your code starts looking like the problem.** Hostel rooms, fee accounts, courses, ID cards — you write classes for the things that actually exist at SRM, and the code reads like the campus works.
> - **This is how real projects are built.** Every Java framework you will touch in placements — Spring, Android, Hibernate — assumes you think in objects. Not "nice to have." Table stakes.

PAUSE & THINK: *With the parallel-arrays approach and 60 students, what happens the day someone drops out and you delete `names[23]` but forget the other two arrays? How would you even notice?*

Try This Code

```
class SrmStudent {
    // DATA that describes one student
    String name;
    String regNo;
    int attendance;

    // BEHAVIOUR that belongs to that student
    void checkExamEligibility() {
        if (attendance >= 75) {
            System.out.println(name + " (" + regNo + ") - " + attendance
                               + "% - Hall ticket released");
        } else {
            System.out.println(name + " (" + regNo + ") - " + attendance
                               + "% - Detained, meet your class advisor");
```

```
        }
    }
}

public class OopIntroDemo {
    public static void main(String[] args) {
        SrmStudent s1 = new SrmStudent();
        s1.name = "Ravi";
        s1.regNo = "RA2311003010123";
        s1.attendance = 82;

        SrmStudent s2 = new SrmStudent();
        s2.name = "Anitha";
        s2.regNo = "RA2311003010124";
        s2.attendance = 68;

        s1.checkExamEligibility();
        s2.checkExamEligibility();
    }
}
```

Output

```
Ravi (RA2311003010123) - 82% - Hall ticket released
Anitha (RA2311003010124) - 68% - Detained, meet your class advisor
```

> **TRY BREAKING IT. Add a third student. Count how many lines you needed. Then go back to the parallel-arrays version at the top of this section and count how many lines *that* would need. That gap is the entire value proposition of OOP, and it only widens as the program grows.**

> **WATCH OUT FOR**
>
> - Treating OOP as extra syntax to memorise for the exam. It is a way of *organising*; the syntax is just how Java spells it.
> - Building a class for a five-line script. A quick calculator does not need one. Judgement, not dogma.

Quick Recap

- OOP bundles data with the behaviour that acts on it.
- An object is a *thing*; a method is something that thing can do.
- You have been using objects since your first `String`.

2. Benefits: Reusability, Security, Extensibility

Three benefits get quoted in every textbook. Let's make each one concrete instead of abstract, using something every SRM student knows painfully well — **the fee portal**.

Reusability

Write the `FeeAccount` class once. SRM has tens of thousands of students. Same class, that many objects. And when you find a bug in the payment logic, you fix it in **one** place and every account is fixed.

Better still: a hostel fee account is *a kind of* fee account with a little extra. Instead of copy-pasting the whole class, you **inherit** — take everything, then add what is different. That is the `extends` keyword.

Security (Encapsulation)

Here is the real question: should any code anywhere be allowed to write

```
ravi.amountPaid = 200000;   // fees cleared! congratulations!
```

Obviously not. So you mark `amountPaid` as `private`. Now the outside world cannot touch it. The only way in is through `pay()` — a method **you** wrote, containing **your** rules.

Think of the fee counter in the admin block. You can hand over money. You cannot walk behind the desk and edit the ledger. That desk is `private`.

Extensibility

Six months later, the college adds an EMI option. With OOP you write a *new* class that extends the old one. The original `FeeAccount` — already tested, already handling every student, already working — is not touched. Working code that stays untouched stays working.

> **WHY THIS MATTERS**
>
> - **Reusability** shrinks both the code you write and the code you have to fix later.
> - **Security** is not about hackers. It is about protecting data from *accidental* misuse — including by you, six months from now, at 2 AM, having forgotten the rules.
> - **Extensibility** is why software survives for years. Requirements always change. OOP lets you grow a system instead of rewriting it.

PAUSE & THINK: *If `amountPaid` were public, where exactly would the "payment must be positive" check live? Now imagine four different developers each writing code that touches fees. How many of them will remember to write that check?*

Try This Code

```
class FeeAccount {
    private String regNo;         // private = the counter you cannot walk behind
    private double totalFee;
    private double amountPaid;

    public FeeAccount(String regNo, double totalFee) {
        this.regNo = regNo;
        this.totalFee = totalFee;
    }

    public void pay(double amount) {
        if (amount <= 0) {                      // THE RULE. Lives in one place.
            System.out.println("Rejected: a payment of Rs " + amount + " makes no sense");
            return;
        }
        amountPaid += amount;
```

```
        System.out.println("Rs " + amount + " received for " + regNo);
    }

    public double getDue() {                    // controlled, read-only access
        return totalFee - amountPaid;
    }
}

// REUSABILITY + EXTENSIBILITY: new ability, zero edits to FeeAccount
class HostelFeeAccount extends FeeAccount {
    public HostelFeeAccount(String regNo, double totalFee) {
        super(regNo, totalFee);
    }

    public void payInTwoInstallments(double amount) {
        pay(amount / 2);      // reusing the inherited, already-tested method
        pay(amount / 2);
    }
}

public class BenefitsDemo {
    public static void main(String[] args) {
        HostelFeeAccount ravi = new HostelFeeAccount("RA2311003010123", 200000);

        ravi.pay(90000);
        ravi.pay(-5000);                  // blocked by the rule inside the class
        ravi.payInTwoInstallments(60000);

        System.out.println("Still due: Rs " + ravi.getDue());
    }
}
```

Output

```
Rs 90000.0 received for RA2311003010123
Rejected: a payment of Rs -5000.0 makes no sense
Rs 30000.0 received for RA2311003010123
Rs 30000.0 received for RA2311003010123
Still due: Rs 50000.0
```

TRY BREAKING IT. In `main`, add the line `ravi.amountPaid = 200000;` and try to compile. Read the error message carefully — that error *is* encapsulation doing its job. Then change `private double amountPaid;` to `public` and watch the protection vanish.

WATCH OUT FOR

- Marking every field `public` because it is faster to type. You have just thrown away the entire security benefit.
- Auto-generating a getter *and* a setter for every field out of habit. If nothing outside should ever change `totalFee`, do not give it a setter.

Quick Recap

- **Reusability:** one class, unlimited objects; inherit instead of copy-pasting.
- **Security:** `private` fields + public methods = rules that cannot be bypassed.
- **Extensibility:** add new classes rather than editing working ones.

3. Class & Object — The Idea That Has to Click

If only one section of this guide sticks, make it this one.

The Analogy

Walk into the SRM hostel office and ask for the **floor plan** of C Block. They will show you a drawing: each room has three beds, one window, one cupboard, a room number.

Now try to sleep in that drawing.

You cannot, obviously. The floor plan is a *description*. Room C-214 is a *real room* — it has an actual number, actual students, and its own actual mess. Room C-507 was built from the exact same plan but is a completely separate room. Someone moving into C-214 does not put a person in C-507.

The floor plan is the class. The room is the object.

Floor plan (Class)	Actual room (Object)
Drawn once	Built many times
No students live in it	Real occupants, real state
Says what a room *has* and *does*	Actually has real values
`class HostelRoom { ... }`	`HostelRoom r = new HostelRoom();`

Other analogies that work just as well: your Reg. No. **format** vs your actual Reg. No.; the exam **question paper** vs the 60 filled answer sheets; a WhatsApp group **description** vs the messages actually in it.

> **WHY THIS MATTERS**
>
> - The class describes; the object exists. That one sentence explains almost every beginner error in this topic — including why you cannot call an instance method without creating an object first.
> - Each object carrying its own copy of the fields is what keeps C-214 and C-507 independent. Those values are called the object's **state**.

PAUSE & THINK: *A class is only a description. So what exactly is happening in the instant you type `new HostelRoom()`?*

Try This Code

```
// THE FLOOR PLAN -- describes every room, is not any particular room
class HostelRoom {
    String roomNo;
    int beds;
    int occupied;

    void allot(String studentName) {
        if (occupied < beds) {
            occupied++;
            System.out.println(studentName + " allotted to room " + roomNo);
        } else {
            System.out.println("Room " + roomNo + " is full - " + studentName
                               + " goes on the waiting list");
        }
    }
}

public class BlueprintDemo {
    public static void main(String[] args) {
        // TWO ACTUAL ROOMS built from the same plan
        HostelRoom room214 = new HostelRoom();
        room214.roomNo = "C-214";
```

```
        room214.beds = 3;

        HostelRoom room507 = new HostelRoom();
        room507.roomNo = "C-507";
        room507.beds = 2;

        room214.allot("Ravi");
        room214.allot("Karthik");
        room507.allot("Meera");

        // Filling C-214 did nothing to C-507. Separate objects, separate state.
        System.out.println("C-214 occupied: " + room214.occupied);
        System.out.println("C-507 occupied: " + room507.occupied);
    }
}
```

Output

```
Ravi allotted to room C-214
Karthik allotted to room C-214
Meera allotted to room C-507
C-214 occupied: 2
C-507 occupied: 1
```

TRY BREAKING IT. Allot a fourth student to C-214 and watch the waiting-list message appear. Then comment out `room507.beds = 2;` and run again — Meera gets waitlisted from an empty room, because an unset `int` is `0`, and `0 < 0` is false. Java never leaves a field as garbage; it fills in `0`, `false`, or `null`.

WATCH OUT FOR

- Saying "the class stores the data." It does not. Each object does. The class only declares *what kind* of data there will be.
- Expecting `String` fields to start as `""`. They start as `null`, which is a very different and much crashier thing.

Quick Recap

- Class = floor plan. Object = the actual room.
- Every object gets its own copy of the instance fields, so objects stay independent.
- Unset fields hold Java's defaults: `0`, `false`, `null`.

4. Class Definition and Syntax

A class has three ingredients. That is genuinely all.

```
class ClassName {
    // 1. FIELDS        -- what each object has
    // 2. CONSTRUCTOR   -- how it gets built
    // 3. METHODS       -- what it can do
}
```

The Constructor — and Why It Matters

Look back at Section 3. We created a room and *then* set `roomNo` and `beds` on separate lines. For a moment, that room existed with no number and no beds. A half-built object.

Now imagine forgetting one of those lines. You have a room object that is silently broken, and nothing will complain until much later.

A **constructor** closes that gap. It runs automatically at the moment of creation, and if you demand values in it, the object simply cannot come into existence incomplete.

Two rules, no exceptions:

1. Same name as the class.
2. **No return type at all** — not even `void`.

If you write no constructor, Java quietly hands you a free no-argument one. The moment you write your own, that free one disappears.

The `this` Keyword

`this` means *"the object this method is currently working on."* You need it when a parameter shares a name with a field:

```
this.code = code;    // this.code = the field    |    code = the parameter
```

Drop the `this` and you write `code = code;` — assigning the parameter to itself. The field stays `null`. It compiles fine. It just doesn't work, which is the worst kind of bug.

Access Modifiers

Modifier	Who can reach it
`public`	Anyone, anywhere
`private`	Only code inside the same class
`protected`	Same package + subclasses
(nothing)	Same package only ("default")

Everyday rule: fields `private`, methods `public`. Deviate only when you can explain why.

Naming and File Rules

- Class names in **PascalCase**: `FeeAccount`, `HostelRoom`.
- A `public` class must sit in a file of exactly the same name — `Course` goes in `Course.java`.
- One file can hold several classes, but only one may be `public`.

PAUSE & THINK: *Every other method in Java must declare a return type. Why is a constructor exempt — what would it even return?*

Try This Code

```
class Course {
    // FIELDS -- private, because nobody outside should rewrite your grade point
    private String code;
    private String title;
    private int credits;
    private int gradePoint;

    // CONSTRUCTOR -- same name as the class, no return type
    public Course(String code, String title, int credits, int gradePoint) {
        this.code = code;             // 'this.code' is the field
        this.title = title;           // plain 'code' is the parameter
        this.credits = credits;
        this.gradePoint = gradePoint;
    }

    // METHODS -- behaviour that belongs to a course
    public int creditPoints() {
        return credits * gradePoint;
    }

    public void printCard() {
        System.out.println(code + " | " + title + " | " + credits
                           + " credits | GP " + gradePoint);
    }
}

public class ClassSyntaxDemo {
    public static void main(String[] args) {
        Course dsa  = new Course("21CSC201J", "Data Structures", 4, 9);
        Course oops = new Course("21CSC203P", "Advanced Programming Practice", 3, 8);

        dsa.printCard();
        oops.printCard();

        System.out.println("Total credit points: "
                           + (dsa.creditPoints() + oops.creditPoints()));
    }
}
```

Output

```
21CSC201J | Data Structures | 4 credits | GP 9
21CSC203P | Advanced Programming Practice | 3 credits | GP 8
Total credit points: 60
```

TRY BREAKING IT — three experiments, and they are all worth doing.

1. Change `public Course(...)` to `public void Course(...)`. It still compiles. But `new Course("21CSC201J", ...)` now fails, because you have turned the constructor into an ordinary method. Read that error until it makes sense.
2. Remove the `this.` from `this.code = code;`. It compiles. Run it and watch `null` appear on your grade card.
3. Add `Course extra = new Course();` — Java refuses, because writing your own constructor removed the free one.

WATCH OUT FOR

- Giving a constructor a return type. Silent, confusing, extremely common.
- Forgetting `this` on a name clash. Also silent. Also extremely common.
- Assuming `new Course()` still works after you have written a constructor with parameters.

Quick Recap

- Class = fields + constructor + methods.
- Constructor: class name, no return type, runs once per object, guarantees a valid object.
- `this` = the current object; it settles field-vs-parameter name clashes.
- Default to `private` fields, `public` methods.

5. Creating and Using Objects

What `new` Actually Does

```
Course dsa = new Course("21CSC201J", "Data Structures", 4, 9);
//    ^        ^     ^
//    |        |     +-- constructor runs: fills the object with values
//    |        +-------- 'new': carves out fresh memory for one object
//    +----------------- reference variable: stores WHERE that object lives
```

Three separate events on one line. That third one is where everyone gets caught.

The Big One: References Are Not Objects

`dsa` does not contain the course. It contains the *address* of the course.

Think of it like the Wi-Fi password for your hostel block, written on a slip of paper. You photocopy the slip and give it to your roommate. There are now two slips — but still **one** network. Your roommate does not get their own private Wi-Fi.

Same thing here:

```
IdCard duplicate = ravi;
```

This does **not** clone the ID card. It copies the address. Two variable names, one single object. Change it through either name and both "see" it — because there is only one thing to see.

You have met this already. Last week, `a == c` was `false` for two Strings with identical text, and you were told to use `.equals()`. Now you know *why*: `==` compares addresses, not contents. Same rule, and it applies to every object you will ever write.

`null` — The Reference to Nothing

`null` means the variable points at nothing at all. Call a method on it and you get a `NullPointerException` — the single most common runtime error in Java, and always the same cause: you asked something that doesn't exist to do something.

Garbage Collection

When no reference points to an object any more, Java's garbage collector reclaims that memory automatically. Unlike C or C++, you never free memory by hand. One less thing to get wrong.

> **PAUSE & THINK:** *If two variables can point to the same object, what happens when you pass an object into a method and that method modifies one of its fields? Does the change survive after the method ends?*

Try This Code

```
class IdCard {
    String name;
    String regNo;
    int libraryBooks;

    IdCard(String name, String regNo) {
        this.name = name;
        this.regNo = regNo;
    }
}

public class ObjectUsageDemo {
    public static void main(String[] args) {
        IdCard ravi      = new IdCard("Ravi", "RA2311003010123");
        IdCard duplicate = ravi;                                  // SAME card, second name
        IdCard karthik   = new IdCard("Ravi", "RA2311003010123");  // separate card, same details
```

```
        duplicate.libraryBooks = 3;      // issued through the "duplicate"

        System.out.println(ravi.libraryBooks);    // 3  -- there was only ever one card
        System.out.println(ravi == duplicate);    // true  -- same object
        System.out.println(ravi == karthik);      // false -- identical data, different object

        IdCard lost = null;              // a card holder with no card in it
        // System.out.println(lost.name);   // NullPointerException
        System.out.println(lost == null);         // true
    }
}
```

Output

```
3
true
false
true
```

Look hard at line 3 of that output. `ravi` and `karthik` have the *exact same name and reg. no.* and `==` still says `false`. That is the Week 2 String lesson returning in its general form.

> **TRY BREAKING IT. Uncomment `System.out.println(lost.name);`. Read the `NullPointerException` and note the line number — this exact error will greet you a hundred more times, and recognising it instantly is a real skill.**

> **WATCH OUT FOR**
>
> - Assuming `objectA = objectB` makes a copy. It gives you a second name for one object.
> - Declaring `IdCard c;` and using it before assigning anything.
> - Reaching for `==` when you meant "same contents." That needs an `equals()` method — which is exactly where you are headed next.

Quick Recap

- `new` allocates, the constructor initialises, the variable stores a reference.
- Copying a reference does not copy the object.
- `null` = points at nothing; using it throws `NullPointerException`.
- Unreachable objects are cleaned up for you.

6. Instance vs Class (static) Members

One Question Decides Everything

For every field you write, ask yourself exactly one thing:

> **Does every object need its own copy, or is there only ever one of these for the whole college?**

Your name? Yours alone. Your reg. no.? Yours alone. Your CGPA? Definitely yours alone.

The university's name? There is one. "SRM Institute of Science and Technology" does not need tens of thousands of copies sitting in memory. Neither does the total admission count — there is one number, and every admission bumps that same number.

Answer "own copy" → **instance member**. Answer "one for everyone" → **static member**.

	Instance member	Static member
Copies	One per object	Exactly one, shared
Declared with	*(nothing special)*	`static`
Accessed via	An object: `ravi.name`	The class: `SrmStudent.campus`
Born when	Every `new`	Once, when the class loads
Good for	Data that differs per object	Shared constants, counters, utilities

The One Rule You Must Remember

A static method cannot touch instance fields or `this`.

Why? A static method belongs to the class, not to any object. If you call `SrmStudent.printTotalAdmissions()` and it tries to print `name` — *whose* name? There is no object involved. The question is meaningless, so Java refuses to compile it.

This is the source of the error every single beginner hits:

```
non-static variable cannot be referenced from a static context
```

And now you know why `main` is `static` — the JVM has to call it *before* any object exists. There is nothing for it to belong to yet.

The reverse is completely fine: an instance method can read static members freely.

> **WHY THIS MATTERS**
>
> - A static field is **shared state**. Change it once and every object sees the new value instantly. Excellent for constants and counters. Dangerous for anything that should differ per object.
> - Java's own constants work this way. `Math.PI` exists once, never changes, and needs no `Math` object — which is why you have never written `new Math()`.

PAUSE & THINK: *Suppose you mark `attendance` as `static` in `SrmStudent`. What happens the moment you set the second student's attendance? Predict it before you test it.*

Try This Code

```
class SrmStudent {
    String name;                                    // INSTANCE: yours alone
    String regNo;                                   // INSTANCE: yours alone
```

```
     static String university = "SRM Institute of Science and Technology";
    static String campus     = "SRM Kattankulathur"; // STATIC: one for everyone
    static int admissionCount = 0;                   // STATIC: shared counter

    SrmStudent(String name) {
        this.name = name;
        admissionCount++;                            // same counter, every time
        this.regNo = "RA23110030101" + admissionCount;
    }

    void printIdCard() {                             // instance method: needs an object
        System.out.println(regNo + " | " + name + " | " + campus);
    }

    static void printTotalAdmissions() {             // static method: called on the class
        System.out.println("Students admitted so far: " + admissionCount);
        // System.out.println(name);   // ERROR -- whose name, exactly?
    }
}

public class StaticDemo {
    public static void main(String[] args) {
        SrmStudent ravi  = new SrmStudent("Ravi");
        SrmStudent meera = new SrmStudent("Meera");

        ravi.printIdCard();
        meera.printIdCard();

        SrmStudent.printTotalAdmissions();     // no object needed

        SrmStudent.campus = "SRM Ramapuram";   // one line, every student transferred
        ravi.printIdCard();
        meera.printIdCard();
    }
}
```

Output

```
RA231100301011 | Ravi | SRM Kattankulathur
RA231100301012 | Meera | SRM Kattankulathur
Students admitted so far: 2
RA231100301011 | Ravi | SRM Ramapuram
RA231100301012 | Meera | SRM Ramapuram
```

One assignment moved both students to another campus — because there was only ever one `campus` to change. That is static in a single line of output.

> **TRY BREAKING IT. Uncomment `System.out.println(name);` inside `printTotalAdmissions()`. The error you get is the one that has confused every Java beginner since 1995 — and you should now be able to explain it in one sentence.**

> **WATCH OUT FOR**
>
> - Slapping `static` on a field to silence a compiler error in `main`. That error is telling you something true. The fix is almost always to create an object, not to share the field across every student in the college.
> - Calling a static member through an object (`ravi.printTotalAdmissions()`). It compiles, but it disguises the fact that no object is involved. Write `SrmStudent.printTotalAdmissions()`.
> - Using static fields as convenient global variables. Works beautifully in a 50-line program. Becomes genuinely unmaintainable in a 5,000-line one.

Quick Recap

- Instance members belong to objects; static members belong to the class.
- Static code cannot see instance fields or `this` — there is no object to see.
- `main` is static because it runs before any object exists.
- Use `static` for counters, constants, and utilities. Not as an escape hatch from errors.

Wrap-Up

Cheat Sheet

Topic	Remember This
What is OOP	Stop describing steps; describe things. Data + behaviour in one unit.
Benefits	Reusability (write once, inherit), Security (`private` + methods), Extensibility (add classes, don't edit working ones).
Class vs Object	Floor plan vs actual room. The class describes; the object exists.
Class Syntax	Fields + constructor (same name, no return type) + methods. `this` = current object.
Objects	`new` allocates, constructor initialises, variable holds a *reference*. Copying a reference is not copying the object.
Instance vs Static	Own copy per object, or one for the whole college? That question decides it.

Test Yourself

Answer from memory first. Whatever you cannot answer tells you exactly which section to reread.

1. Explain the difference between a class and an object using an SRM example that is *not* the hostel room.
2. Why does `private` protect data better than "being careful"?
3. What are Java's defaults for an unset `int`, `boolean`, and `String` field?
4. Why does a constructor have no return type?
5. After `IdCard b = a;`, how many objects exist? What happens when you change `b.libraryBooks`?
6. Why can `main` not use an instance field directly? Answer in one sentence.
7. If `campus` is static and one object changes it, what do all the other objects see?
8. When would making a field `static` be an actively *bad* idea?